

Final Project: DC motor speed controller using 8051 MCU

Group 7 - Monday

December 3rd, 2025

Shaik Ikramuddin Jeelani – 3 [REDACTED] 4

Haneen Al-Baidhani - 3 [REDACTED] 4

Abhi Tharmapala - 3 [REDACTED] 8

SCHOOL OF MECHATRONIC SYSTEMS ENGINEERING
SIMON FRASER UNIVERSITY

Introduction:

This project implements a closed-loop speed control system for a DC fan using an 8051 microcontroller. A three-bit DIP switch is used to select a desired speed level, which is converted into a reference RPM. The actual fan speed is measured using a tachometer signal connected to the microcontroller and timed with an internal hardware timer. A proportional control algorithm compares the reference speed with the measured speed and adjusts the PWM duty cycle driving the fan so that the motor speed converges toward the selected setpoint. At the same time, the measured RPM is shown on a three-digit seven-segment display, allowing the user to observe how the PWM signal and feedback control affect the fan's steady-state speed and response.

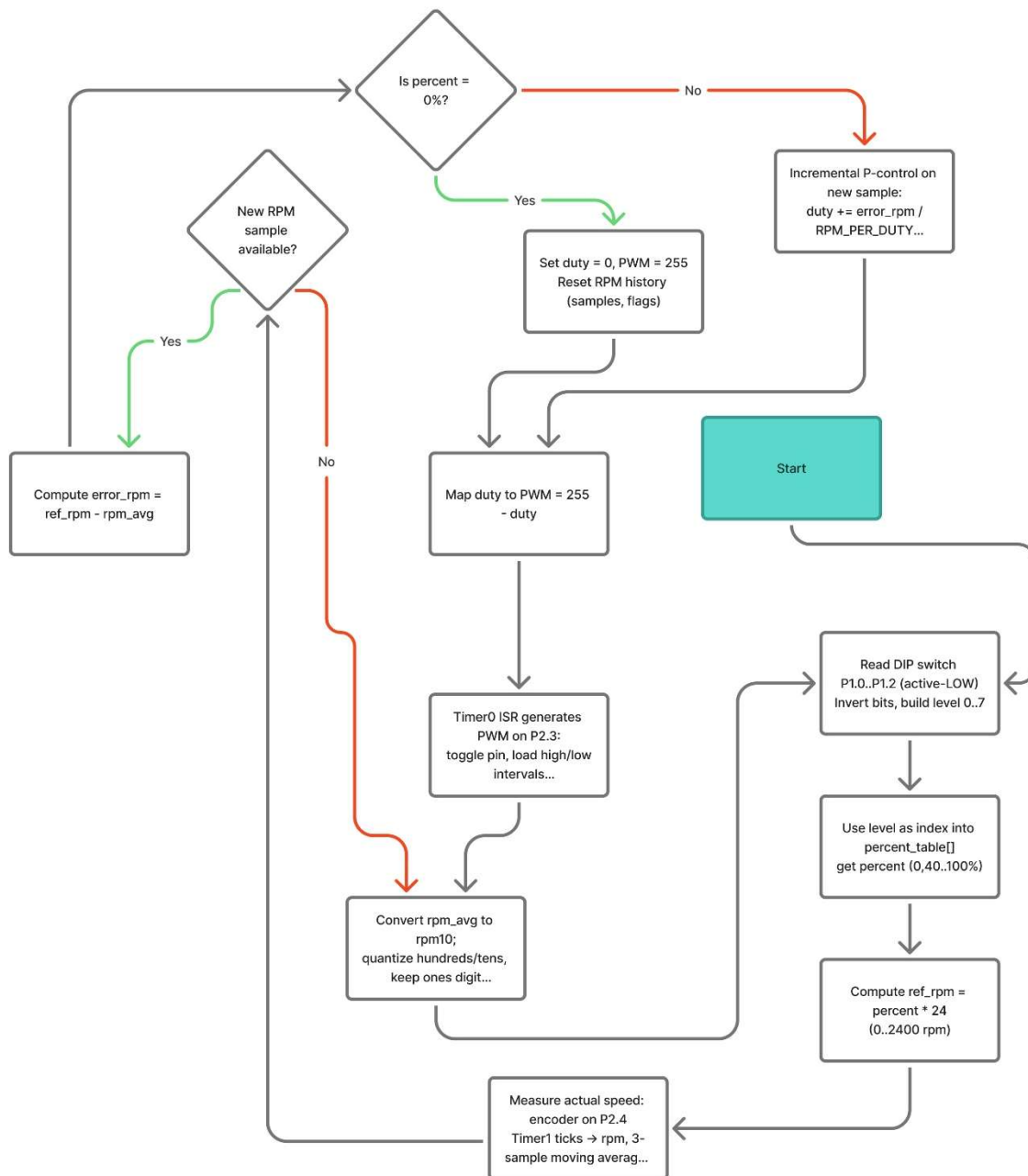
Problem Statement:

The main problem is to adjust the PWM duty cycle so that the measured fan speed tracks the selected setpoint despite non-ideal motor behavior, and to display the resulting speed on a three-digit seven-segment display. In other words, we must integrate digital input, timer-based speed measurement, feedback control, and PWM generation into a single working embedded control system.

Apparatus:

1. Hardware:
 - 1.1. Dip Switch 1x
 - 1.2. 7 Segment CA 3x
 - 1.3. IRF540 1x
 - 1.4. 2N3904 3x
 - 1.5. 40 pin 8051 1x
 - 1.6. AUB0812L-9X41 1x
 - 1.7. Resistors 6x
2. Software:
 - 2.1. Keil uVision5
 - 2.2. 8051FLASH
 - 2.3. Visual Studio Code
 - 2.4. ChatGPT
 - 2.5. SimulIDE

Design Process - Flow Diagram:



Design Process - Explanation:

1) Read speed command from DIP switch

The three-bit DIP switch connected to P1.0–P1.2 (active low) is read in the main loop. These bits are combined into a 3-bit value level (0–7), which is then used as an index into `percent_table[]` to select the desired fan speed in percentage (0, 40, 50, 60, 70, 80, 90, or 100%).

2) Convert percentage to reference speed (RPM)

The selected percentage is converted into a reference speed `ref_rpm` in revolutions per minute. This is done with a simple linear scaling, where 0–100% corresponds to 0–2400 rpm. This reference value represents the target speed that the controller will try to maintain.

3) Measure actual speed using encoder and Timer1

The fan's tachometer signal is connected to P2.4. Timer1 is configured as a free-running 16-bit counter. On every rising edge of the encoder signal, the code reads the current Timer1 value, subtracts the previous value to find the period in timer ticks, and converts this into an instantaneous RPM using the known clock frequency. A 3-sample moving average is then applied to obtain a smoother measurement, stored as `rpm_avg`.

4) Compute speed error for feedback control

Whenever a new RPM measurement is available, the controller computes the speed error as $error_rpm = ref_rpm - rpm_avg$. This error indicates whether the fan is running too slowly (positive error) or too fast (negative error) relative to the DIP-selected speed.

5) Update the PWM duty value in software

For non-zero reference speeds, an incremental proportional control law is used to update the duty variable (0–255) each time a new RPM sample is processed. The error is divided by a tuning constant `RPM_PER_DUTY`, and this term is added to duty. Very small errors inside a chosen deadband are ignored, and the duty change per update is limited, which reduces oscillation around the setpoint. The result is then clamped between 0 and 255. If the selected speed is 0%, the duty is forced to zero and the speed history variables are reset.

6) Map duty to the PWM control variable

The duty value is inverted and written into the global PWM variable as $PWM = 255 - duty$, because the Timer0 interrupt routine is implemented in an inverted form (a larger PWM value corresponds to a longer high time in the ISR calculations).

7) Generate the PWM waveform using Timer0 and interrupts

Timer0 is configured in 16-bit mode with interrupts enabled. On every Timer0 overflow, the ISR toggles the PWM output pin P2.3 and reloads TH0 and TL0 based on the current PWM value. One branch calculates a delay proportional to $PWM * PWM_Freq_Num$ for the high interval, and the other uses $(255 - PWM) * PWM_Freq_Num$ for the low interval. This keeps the overall period roughly constant while changing the ratio of high time to low time, creating a PWM signal whose duty cycle is set by the controller.

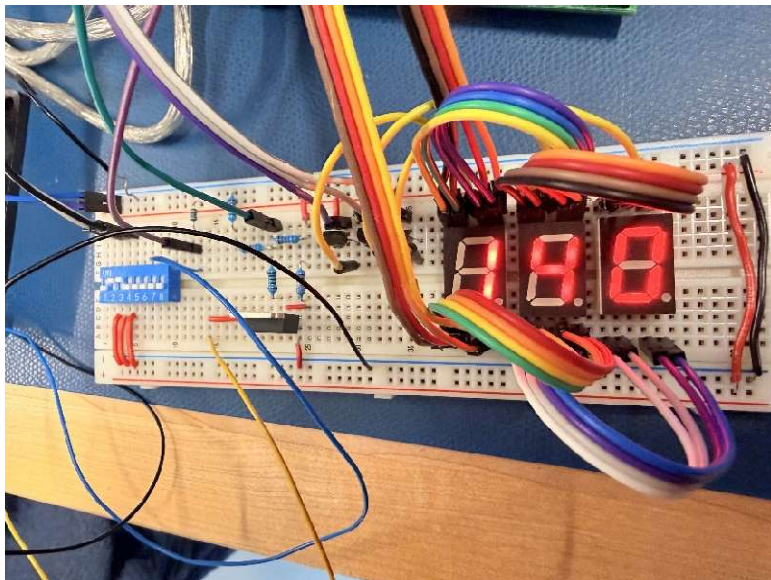
8) Display the measured speed on the seven-segment LEDs

The measured speed `rpm_avg` is converted to $RPM/10$ (`rpm10`), and then to three decimal digits. To make the display more stable, the hundreds and tens digits are taken from a quantized value (`rpm10_base10`), while the ones digit uses the full `rpm10` and is allowed to fluctuate. These three digits are stored in `disp_digits[]` and shown on a multiplexed three-digit seven-segment display using P0 for segments and P2.0–P2.2 as digit enables, updated repeatedly by `refresh_display()` in the main loop.

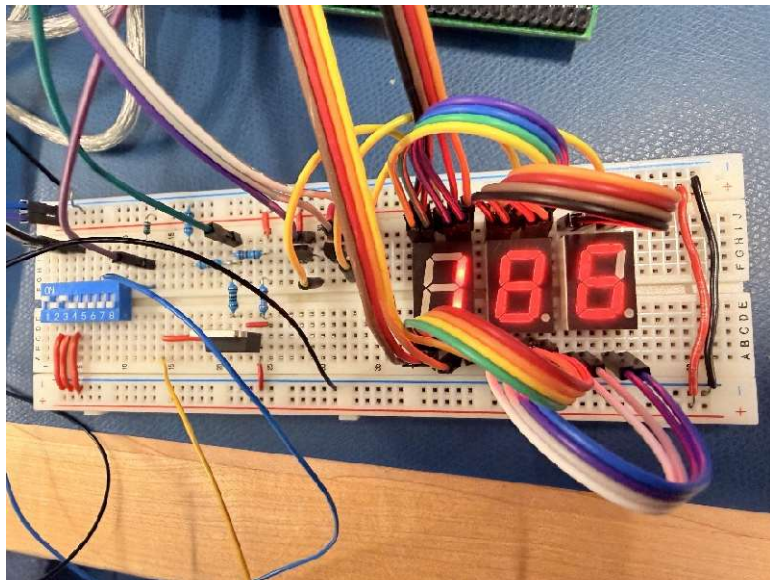
Conclusion:

This project successfully implemented a closed-loop PWM speed control system for a DC fan using the 8051: the DIP switch selected the reference speed, the encoder feedback and proportional controller adjusted the PWM duty on P2.3, and the measured RPM was shown on the three-digit display. During implementation we observed some practical issues, such as noise and jitter in the encoder signal, steady-state error due to limited controller gain resolution, and small fluctuations in the displayed RPM caused by sampling and averaging, but the system still achieved clear and repeatable tracking of multiple speed setpoints.

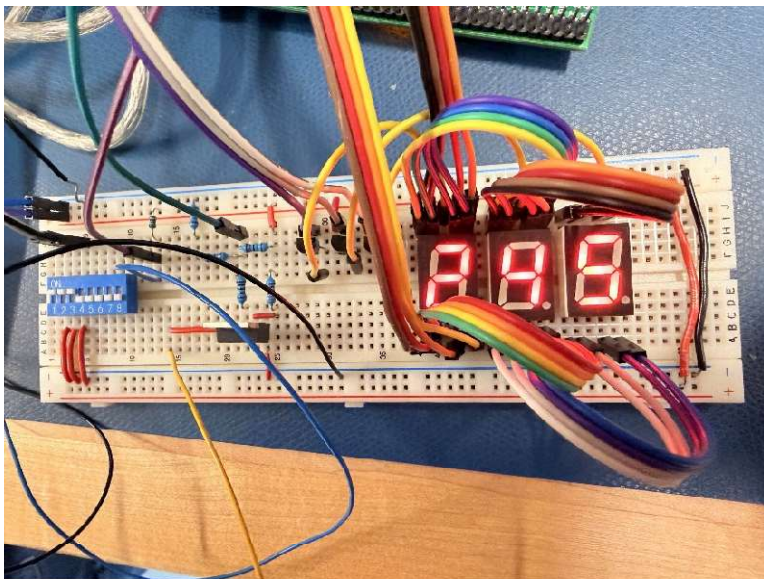
Pictures:



Input: 100, Desired Output: 144, Actual Output: 140



Input: 101, Desired Output: 192, Actual Output: 186



Input: 111, Desired Output: 240, Actual Output: 245